Project Abstract

Track B: Open-Source Fuzzing Campaign

Field	Detail
Target	libsndfile (C audio file I/O library)
Team Lead	Francis Njoku
Members	Francis Njoku, Armando Mendoza Sanchez
Course	Software Security
Date	September 13, 2026

1. Problem Definition

Audio file parsers are a classic memory-safety liability: they accept fully attacker-controlled binary input, they implement dozens of legacy container and codec formats, and they are written in C with heavy pointer and buffer arithmetic.

libsndfile is the de facto standard library for reading and writing audio files on Linux. It is linked into Audacity, Ardour, PulseAudio/PipeWire tooling, SoX, Blender, numerous game engines, and a long tail of embedded and scientific software. A single out-of-bounds write reachable from sf_open() is therefore reachable from “the user opened a .wav someone sent them.”

The library has a documented history of exactly this class of bug, including memory-safety CVEs in its FLAC, AIFF, and MS-ADPCM WAV decoding paths. The mainstream formats (WAV/PCM, AIFF, and basic FLAC) have since received sustained fuzzing attention through OSS-Fuzz and are comparatively hardened.

The problem we intend to address is the neglected-format surface. libsndfile supports a substantial set of rarely-exercised legacy containers (CAF, Sound Designer II, W64, MAT4/MAT5, PVF, HTK, AVR, and the Ogg/Vorbis paths) whose parsers are reached only when the file’s magic bytes match. A corpus-driven fuzzer that starts from common .wav seeds will almost never mutate its way into these branches, so coverage there remains thin even after years of continuous fuzzing. These parsers are old, lightly maintained, and still reachable by default from the same public API used by every downstream application.

Our thesis: targeted, format-aware fuzzing of libsndfile’s minor container parsers will reach code that broad campaigns have not, and will surface real memory-safety defects.

2. Proposed Approach

1. Baseline and measure. Build libsndfile from source with clang, AddressSanitizer and UndefinedBehaviorSanitizer, and -fsanitize=fuzzer. Reproduce the upstream harness, run it briefly, and collect a coverage baseline with llvm-cov. This gives us hard evidence of which parsers are actually cold, rather than assuming it.

2. Write our own harnesses. A primary LLVMFuzzerTestOneInput harness drives the in-memory virtual I/O API (sf_open_virtual), so no filesystem round-trip is needed and execution stays fast. Then per-format harnesses that pre-seed a valid container header and let the fuzzer mutate only the body, forcing the target parser to be entered on every execution.
3. Engineer the corpus. Generate minimal valid files in each target format (using libsndfile itself, ffmpeg, and sox), minimize with cmin, and write format dictionaries of chunk identifiers and magic constants so the mutator can produce structurally plausible inputs.
4. Run a real campaign. Multi-core libFuzzer and AFL++ runs over multiple days, with coverage tracked over time. We will report executions per second, cumulative edge coverage, and per-format coverage deltas against the baseline.
5. Triage and root-cause. Minimize each crashing input, reproduce under a debugger, identify the exact faulting line and the defect class (out-of-bounds read or write, use-after-free, integer overflow, uninitialized read), and determine reachability from the public API.
6. Fix and disclose. Write patches where we can, verify the crash no longer reproduces and that the test suite still passes, and report findings upstream following coordinated-disclosure practice.

3. Project Plan

Weeks	Milestone	Deliverable
1-2	Build environment, sanitizer build, baseline coverage report	Reproducible build scripts; baseline coverage numbers
3-4	Primary virtual-I/O harness; first short campaign; corpus v1	Working harness in repo; initial coverage plot
5-6	Per-format harnesses and dictionaries for selected minor formats	Format-targeted harnesses; corpus v2
7-9	Sustained multi-day campaign; continuous coverage tracking	Campaign logs, coverage-over-time data, crash inventory
10-11	Crash triage, deduplication, root-cause analysis	Per-crash writeups with minimized proof-of-concept inputs
12-13	Patch development, regression verification, upstream reporting	Patches and verification; disclosure record
14-15	Final paper, code cleanup, presentation	Final paper, clean repository, presentation

Contingency. The campaign may produce no new crashes, which is a legitimate outcome for a hardened target rather than a project failure. If that happens, the deliverable pivots to a rigorous negative result: measured coverage gains over the baseline harness, an analysis of why the remaining uncovered branches are unreachable or already guarded, and a comparison of libFuzzer versus AFL++ effectiveness on structured binary input. We will decide this pivot at the Week 9 checkpoint so there is time to execute it properly.

4. Expected Outcome

By the end of the semester we will deliver:

- A public repository containing our harnesses, build scripts, corpora, and dictionaries, reproducible by a third party.
- Quantitative campaign results: total executions, executions per second, cumulative edge coverage, and per-format coverage improvement over the baseline harness.
- A triaged crash inventory, each entry with a minimized proof-of-concept input, stack trace, root-cause analysis, and defect classification.
- Patches for defects we successfully root-cause, plus verification that they resolve the crash without regressing the test suite.
- A final paper and presentation covering methodology, results, and what we learned about where fuzzing effort actually pays off.

Our target metric: a measurable coverage increase in at least three minor-format parsers relative to the baseline harness, and at least one root-caused, reproducible defect or a defensible negative result explaining its absence.

5. Team Members and Contributions

Member	Role	Contribution
Francis	Harness engineering	Primary virtual-I/O harness, sanitizer build configuration, repository and CI setup. Submits deliverables, serves as instructor point of contact, tracks milestone completion.
Francis	Corpus and campaign	Seed-corpus generation and minimization, format dictionaries, campaign orchestration across cores, execution logging.
Armando	Coverage and measurement	llvm-cov instrumentation, baseline and per-format coverage analysis, coverage-over-time plots, comparative libFuzzer and AFL++ evaluation.
Armando	Triage and remediation	Crash minimization and deduplication, debugger-based root-cause analysis, defect classification, patch development and regression verification.

Roles indicate primary ownership, not isolation. All four members will run campaigns and participate in triage, so that each can explain the full pipeline at a high level and their own component in depth during the technical interview.

6. Why This Scope Is Realistic

libsndfile is roughly 60,000 lines of C with a small, stable public API and a straightforward Autotools/CMake build that compiles with sanitizers in minutes, not hours. Unlike ffmpeg or OpenSSL, one team can hold the whole attack surface in their heads. The parsers we are targeting are each a few hundred lines, which makes root-cause analysis tractable within a semester rather than open-ended. The work also scales gracefully: the harness alone is a functioning deliverable by Week 4, and everything after that adds depth rather than being load-bearing for a passing result.

Our team used an AI assistant (Claude) as a drafting and thinking aid for this abstract: talking through candidate fuzzing targets, pressure-testing whether libsndfile was a realistic scope for one semester, and tightening wording on sections we had already outlined. The technical direction is ours, including the decision to target neglected legacy container formats rather than the heavily fuzzed WAV and FLAC paths. We treated its output as unverified and removed a set of CVE numbers we could not confirm against the NVD. For the implementation phase we expect similar use, mostly harness scaffolding and sanitizer output explanations, while all harness logic, corpus design, and crash triage will be done and understood by team members directly.